Indexation avec table de hachage

Présentation du contexte

Vous développez un système de stockage que l'on retrouve dans certains SGBD du type NoSQL, notamment ceux basés sur le modèle de données clé-valeur. Ce système est pertinent lorsque plusieurs valeurs sont associées à une même clé — c'est par exemple le cas si l'on souhaite compter le nombre d'occurrences d'un élément identifié par une clé unique, ou obtenir la valeur la plus récente associée à une clé donnée.

Dans la Figure 1, on retrouve 3 clés (`abc`, `def` et `ghi`). Il y a 2 valeurs différentes (`def:732756769*******` et `def:54434194********`) pour la clé `def`. Dans cette même figure, on considère que la ligne contenant les valeurs (sous le tableau) représente le fichier persisté sur disque.

N.B. : la clé (`def`) a été intégrée dans la valeur pour simplifier les explications sur le modèle de stockage.

L'objectif de l'approche de stockage est de garder un archivage de toutes les données et de pouvoir retrouver efficacement la dernière valeur d'une clé donnée. Pour cela, il faut une solution de stockage de fichiers sur disque **ET** une solution d'indexation. L'index sera de préférence en mémoire principale (plus efficace que sur disque en termes d'entrées/sorties) : l'index est gardé en mémoire, et les données sont persistées sur disque (dans des fichiers, pour l'archivage).

L'approche de stockage consiste à ajouter des données à la fin d'un fichier (approche qualifiée d'*append-only* en anglais). Une stratégie d'indexation simple correspond alors à conserver une table de hachage en mémoire dans laquelle chaque clé correspond à un décalage d'octets (*offset*) dans le fichier de données, c'est-à-dire l'endroit où la valeur peut être trouvée.

Dans la suite, on considère que :

- les enregistrements (les valeurs des paires clé/valeur) sont d'une taille fixe (**20 octets** dans notre exemple) ;
- on peut avoir plusieurs entrées pour une même clé.

Dans la Figure 1, le tableau correspond à la table de hachage ; on y observe 3 clés (`abc`, `def` et `ghi`). Les enregistrements sont stockés les uns à la suite des autres dans un fichier (souvent nommé *log segment*).

Figure 1 : Indexation avant compactage

clé	offset
abc	0
def	40
ghi	60

```
offset:   0                   20                  40                  60
fichier:  abc:-765768601******|def:732756769*******|def:54434194********|ghi:-1
```

Dans le contexte de l'exemple de la Figure 1, on considère que le prochain enregistrement aura la clé `mno` et la valeur `mno:681147641*******`. Cette valeur sera stockée à la position 80 (car chaque enregistrement est de taille fixe 20 et que le précédent enregistrement était à la position 60), et sa clé sera insérée dans la table de hachage avec `mno` pour la clé et `80` pour l'offset.

Après un certain temps, on se retrouve avec de gros fichiers qui contiennent des données rarement requêtées (car obsolètes) ; par exemple, pour la clé `def`, la valeur `def:`

`732756769*******` n'est pas la plus récente et il n'y a donc pas d'index permettant d'y accéder rapidement. Dans ce cas, cela fait sens de **compacter** les fichiers après un certain temps. L'impact du compactage est mis en évidence en le lançant sur notre exemple ; on obtient alors une nouvelle version de l'index, présentée dans la Figure 2.

Figure 2 : Indexation après compactage

clé	offset
abc	0
def	20
ghi	40

```
offset:   0                    20                      40
fichier:  abc:-765768601******|def:54434194********|ghi:-1945455653*****
```

Il ne reste qu'un seul enregistrement dans l'index avec la clé `def` (`def:54434194********`), qui est le plus récent, et la table de hachage est à jour. Le nouvel index, résultat de ce compactage, est prêt à recevoir de nouveaux enregistrements.

Le fichier avant compactage est archivé sur disque, et il sera donc possible de rechercher des valeurs dans celui-ci — par exemple pour obtenir l'historique de l'évolution d'une clé dans le temps.

Stratégie de compactage

On adoptera la solution basée sur le rapport entre le nombre d'enregistrements **pertinents** (une seule occurrence par clé distincte : la plus récente) et le nombre total d'enregistrements **non-pertinents** (toutes les occurrences obsolètes) dans le fichier. Si ce rapport est strictement inférieur à un seuil préfixé (par vous-même, et paramétrable) alors le système lance le compactage.

Exemple : avec la suite des 13 enregistrements suivants (dans l'ordre d'arrivée) :

```
abc1, ghi1, abc2, def1, ghi2, def2, def3, abc3, abc4, mno1, ghi3, ghi4, abc5
```

- clés distinctes : `abc`, `ghi`, `def`, `mno` → **4 enregistrements pertinents** (la dernière occurrence de chaque clé)
- enregistrements non-pertinents : 13 – 4 = **9**
- rapport pertinents / non-pertinents = 4 / 9 ≈ **0.44**

Si le seuil est fixé à 0.5, alors il faut lancer le compactage (car 0.44 < 0.5). Sinon, le système peut continuer à accepter des enregistrements sans lancer le compactage.

Requêtage

Vous devez également permettre le requêtage du système de stockage depuis l'index. Dans la Figure 1, le requêtage sur la clé `def` doit retourner la valeur `def:54434194********`, la valeur de la clé `def` la plus récente.

Travail à faire

Vous devez implémenter ce système de stockage en Scala en utilisant autant que possible une programmation de type fonctionnelle (donc des fonctions pures, des variables non mutables). Vous organisez votre code de la manière qui vous semble la plus pertinente, mais en adoptant les grandes lignes d'un développement orienté programmation fonctionnelle.

Pour simplifier votre développement, on considère que la taille des enregistrements est fixe.

Paramétrage

Dans un `object` Scala, vous spécifiez les paramètres de votre programme, à minima :

- le seuil d'activation du compactage,
- la taille des enregistrements.

Dans l'objectif de générer aléatoirement des jeux de données, vous pourrez également enregistrer les paramètres supportant cette génération, par exemple :

- le préfixe des fichiers à enregistrer à la suite d'un compactage,
- les valeurs des clés possibles,
- le nombre d'enregistrements à générer lors d'une expérimentation.

Rendu

Vous rendrez votre projet sur le site de e-learning en y incluant :

- le code source,
- le résultat d'au moins une expérimentation, qui devra mettre en évidence :
    1. quand le compactage a été effectué,
    2. l'état de l'index avant et après compactage,
    3. l'état du fichier enregistré sur disque,
- le résultat de requêtes :
    1. obtenant la valeur la plus récente d'une clé,
    2. ne trouvant pas une clé dans l'index.